

**LAPORAN PRAKTIKUM
PEMROGRAMAN BERBASIS FRAMEWORK**

WEEK 7: API Routes pada Next.js dan Integrasi Firebase (Fullstack Next.js)



Disusun Oleh :
Ghoffar Abdul Ja'far 2341720035/TI3B

**JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI MALANG
2025/2026**

JOBSHEET PRAKTIKUM

Mata Kuliah : Pemrograman Framework

Program Studi : D4 (Sarjana Terapan)

Topik Praktikum : API Routes pada Next.js dan Integrasi Firebase (Fullstack Next.js)

Alokasi Waktu : 4 × 50 menit

A. Tujuan Praktikum

Setelah menyelesaikan praktikum ini, mahasiswa mampu:

1. Memahami konsep API Routes pada Next.js.
 2. Membuat API sederhana menggunakan Next.js.
 3. Mengirim response JSON dengan status code.
 4. Mengambil data API di sisi frontend menggunakan fetch.
 5. Mengintegrasikan Firebase Firestore sebagai database.
 6. Mengelola environment variable (.env.local).
 7. Menampilkan data dinamis dari database ke halaman web.
-

B. Dasar Teori Singkat

1. API Routes di Next.js

Next.js tidak hanya framework frontend, tetapi juga dapat berfungsi sebagai backend melalui API Routes.

File API diletakkan di:

- pages/api/

Contoh:

- pages/api/hello.js

Endpoint dapat diakses melalui:

- /api/hello
-

2. Firebase Firestore

Firebase adalah layanan backend dari Google yang menyediakan:

- Database (Firestore / Realtime DB)
- Authentication

- Storage (File)

Firestore bersifat NoSQL dan cocok untuk aplikasi modern.

C. Alat dan Bahan

Perangkat Lunak

- Node.js (LTS)
- NPM
- Visual Studio Code
- Browser
- Akun Google (untuk Firebase)

Prasyarat

- Project Next.js (Pages Router) sudah tersedia
 - Aplikasi dapat dijalankan (npm run dev)
-

D. Langkah Kerja Praktikum

Langkah 1 – Menjalankan Project

- npm run dev

Akses:

- <http://localhost:3000>
-

Langkah 2 – Membuat API Produk

1. Buat file pada pages/api/produk.js



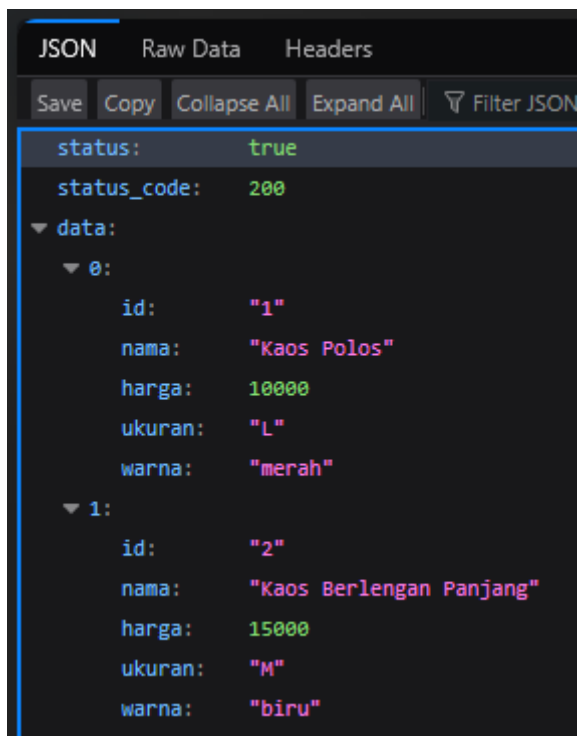
2. Tambahkan data statis:

```

src > pages > api > TS produk.ts > handler > data
1  import type { NextApiRequest, NextApiResponse } from "next";
2
3  type Data = {
4    status: boolean;
5    status_code: number;
6    data: {
7      id: string;
8      nama: string;
9      harga: number;
10     ukuran: string;
11     warna: string;
12   }[];
13 };
14
15 export default function handler(
16   req: NextApiRequest,
17   res: NextApiResponse<Data>,
18 ) {
19   const data = [
20
21     {
22       id: "1",
23       nama: "Kaos Polos",
24       harga: 10000,
25       ukuran: "L",
26       warna: "merah",
27     },
28     {
29       id: "2",
30       nama: "Kaos Berlengan Panjang",
31       harga: 15000,
32       ukuran: "M",
33       warna: "biru",
34     },
35   ];
36   res.status(200).json({ status: true, status_code: 200, data: data });
37 }

```

3. Akses: <http://localhost:3000/api/produk>



Langkah 3 – Fetch Data API di Frontend

1. Buka pages/product/index.tsx

- Modifikasi kode
 - Tambahkan useEffect() dan comment useEffect untuk isLogin

```
1 import { useRouter } from "next/router"; // Remove this unused import of 'useRouter'.
2 import { useEffect, useState } from "react";
3
4 type ProductType = {
5   id: string;
6   nama: string;
7   harga: number;
8   ukuran: string;
9   warna: string;
10 };
11
12 const ProdukPage = () => {
13   // const [isLogin] = useState(false); // Remove this commented out code.
14   // const { push } = useRouter();
15
16   const [products, setProducts] = useState<ProductType[]>([]);
17
18   // useEffect(() => { // Remove this commented out code.
19   //   if (!isLogin) {
20   //     push("/views/auth/login");
21   //   }
22   // }, [isLogin, push]);
23
24   useEffect(() => {
25     fetch("/api/produk")
26       .then((response) => response.json())
27       .then((responseData) => {
28         // console.log("Data produk:", responseData.data); // Remove this commented out code.
29         setProducts(responseData.data);
30       })
31       .catch((error) => {
32         console.error("Error fetching produk:", error);
33       });
34   }, []);
35
36   return (
37     <div>
38       <h1>Daftar Produk</h1>
39       <div key={product.id}>
40         <h2>{product.nama}</h2>
41         <p>Harga: {product.harga}</p>
42         <p>Ukuran: {product.ukuran}</p>
43         <p>Warna: {product.warna}</p>
44       </div>
45     </div>
46   );
47 }
48
49 export default ProdukPage;
```

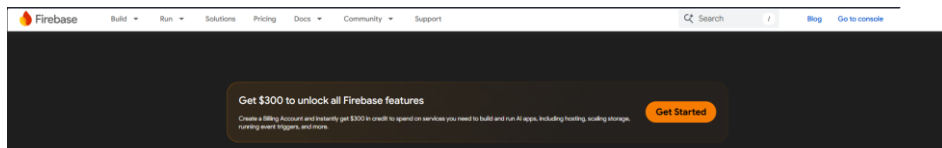
2. Jalankan browser <http://localhost:3000/produk>



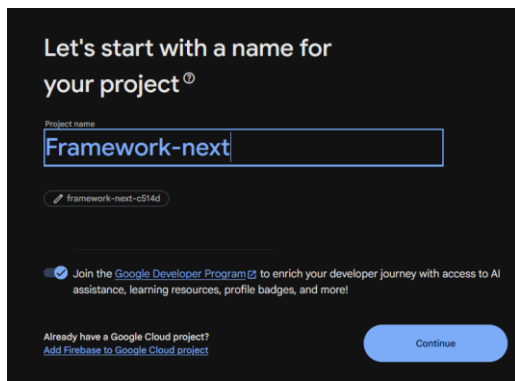
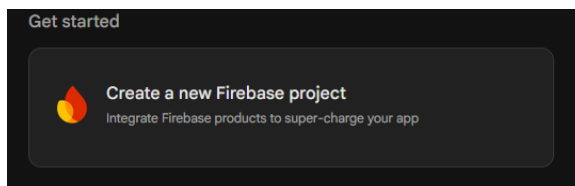
E. Integrasi Firebase

Langkah 5 – Setup Firebase

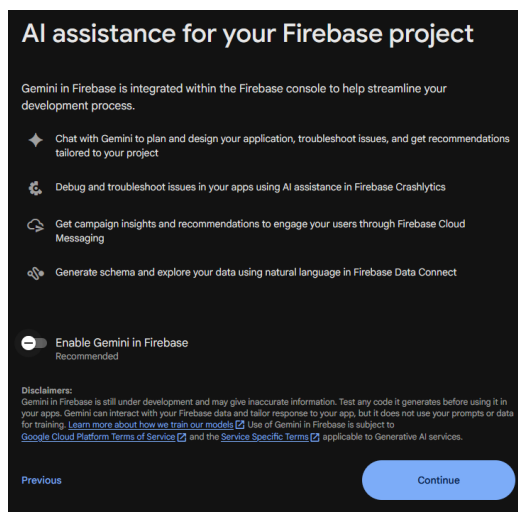
1. Buka Firebase Go To Console (login dengan login google)



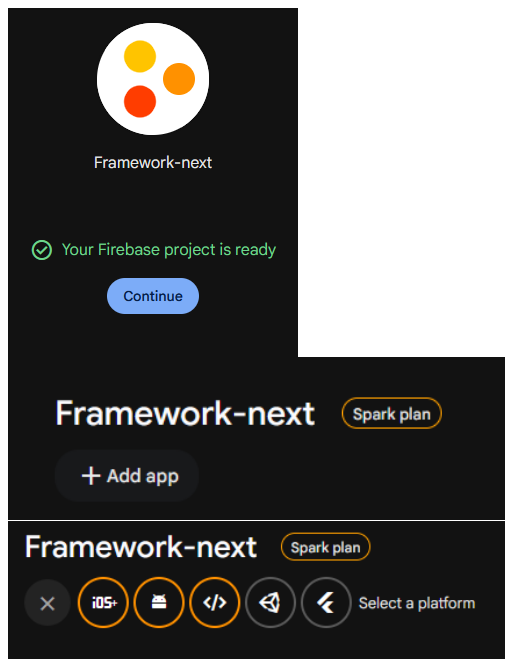
2. Buat project baru



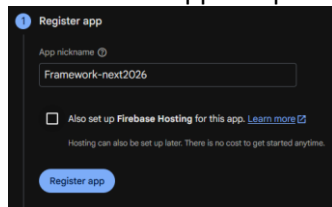
Note : jangan lupa select parent resource nya



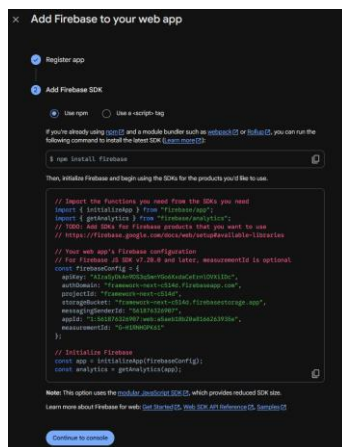
Note : klik create project dan disable google analytics



Note klik add app dan pilih web

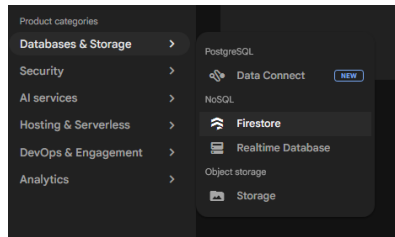


Note : klik register app

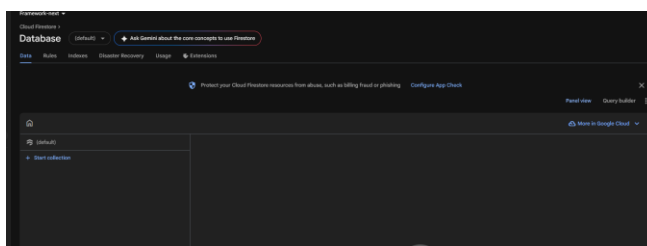
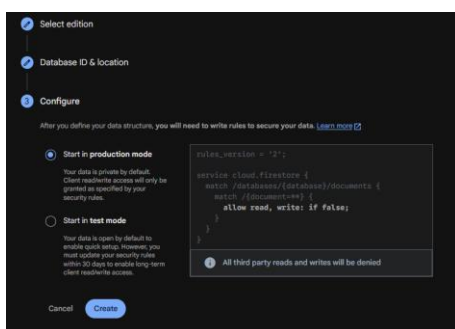
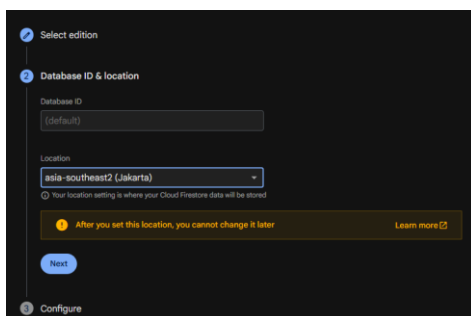
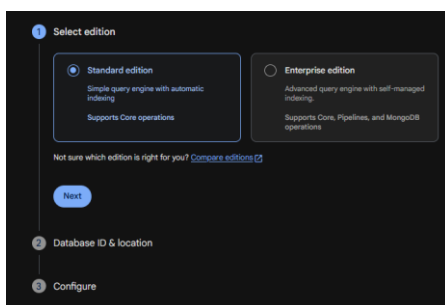
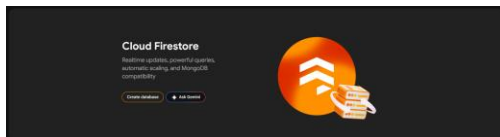


Note : klik continue to console

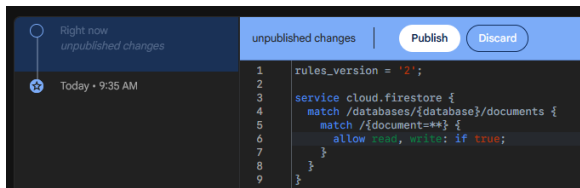
3. Aktifkan Firestore Database



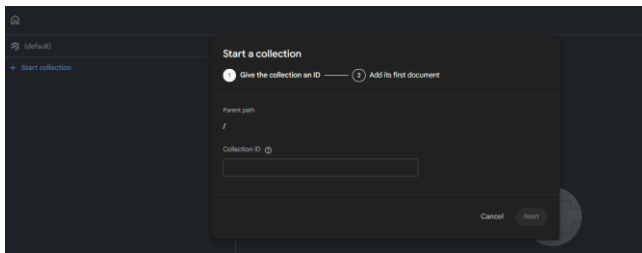
Note : klik create database



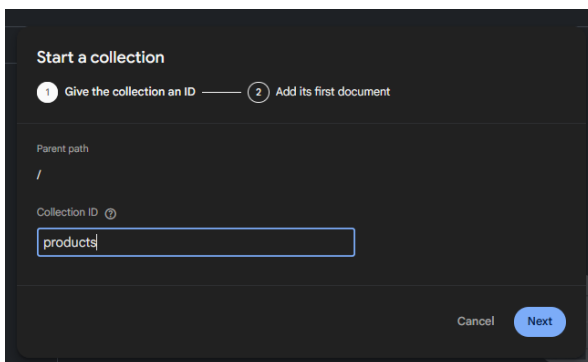
Rubah rulesnya jadi true dan klik publish



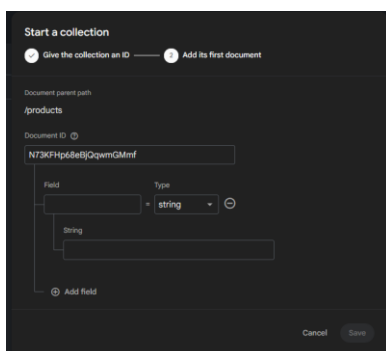
4. Buat collection:



5. Products



Gunakan auto-id



6. Tambahkan field:

Start a collection

Give the collection an ID **1** Add its first document **2**

Document parent path
/products

Document ID
N73KFHp68BjQqwmGMmf

Field Type

name string

String
baju polo

Field Type Value

price int64 100000

Field Type

size string

String
M

+ Add field

Cancel Save

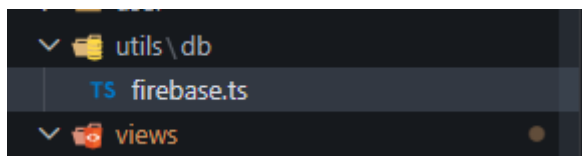
products	products	products
+ Start collection	+ Add document	+ Start collection
products	XNPHpDySPang_3d.P	+ Add field
		name: "baju polo"
		price: 100000
		size: "M"

Langkah 6 – Install Firebase

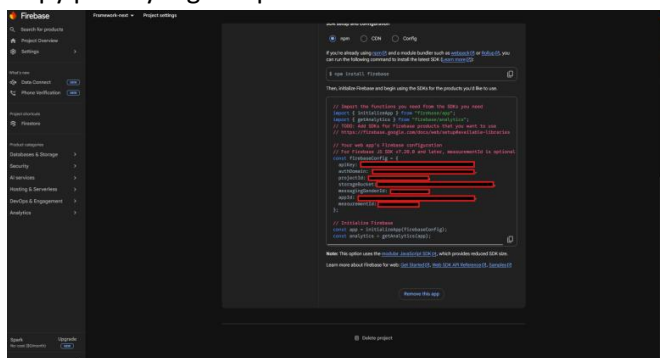
1. npm install firebase

```
PS E:\Vuln\Vuln\Semester 6\Programan Berbasis Framework\urakitu\my_app> npm install firebase
added 79 packages, and audited 533 packages in 1m
189 packages are looking for funding
  run `npm fund` for details
4 vulnerabilities (2 moderate, 2 high)
  To address issues that do not require attention, run:
    npm audit fix
  To address all issues, run:
```

2. Buat folder dan file ts pada pages utilis/db/firebase.ts

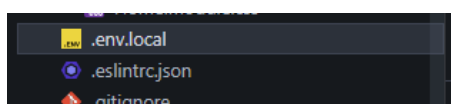


3. Copy paste yang ada pada kotak merah ke file firebase.ts



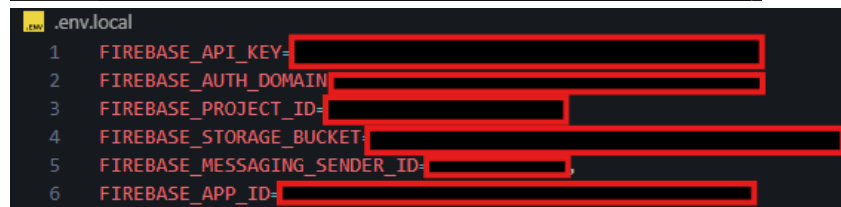
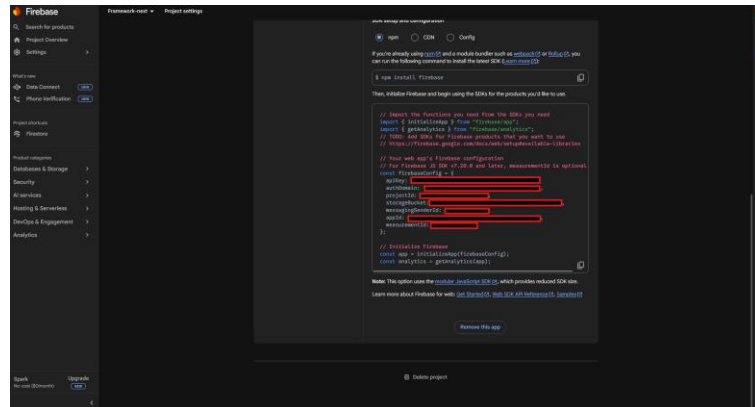
Langkah 7 – Konfigurasi Environment Variable agar credensial firebase tidak dapat dilihat saat dipush di repository

1. Buat file: .env.local



2. Modifikasi file env

- FIREBASE_API_KEY=xxxx, FIREBASE_AUTH_DOMAIN=xxxx, FIREBASE_PROJECT_ID=xxxx, FIREBASE_STORAGE_BUCKET=xxxx, FIREBASE_MESSAGING_SENDER_ID=xxxx, FIREBASE_APP_ID=xxxx
- Isikan sesuai dengan yang ada pada (tanpa koma dan petik dua)



Langkah 8 – Konfigurasi Firebase

1. Modifikasi firebase.ts

```
const firebaseConfig = {
  apiKey: process.env.FIREBASE_API_KEY,
  authDomain: process.env.FIREBASE_AUTH_DOMAIN,
  projectId: process.env.FIREBASE_PROJECT_ID,
  storageBucket: process.env.FIREBASE_STORAGE_BUCKET,
  messagingSenderId: process.env.FIREBASE_MESSAGING_SENDER_ID,
  appId: process.env.FIREBASE_APP_ID
};
```

Langkah 9 – Ambil Data dari Firestore

1. Buat file

- ulits/db/servicefirebase.ts

utils\db	
TS firebase.ts	U
TS servicefirebase.ts	U

- modifikasi file servicefirebase.js

```
src > pages > utils > db > TS servicefirebase.ts > ...
1 import { getFirestore, collection, getDocs } from "firebase/firestore";
2 import app from "../firebase";
3
4 const db = getFirestore(app);
5
6 export async function retrieveProducts(collectionName: string) {
7   const snapshot = await getDocs(collection(db, collectionName));
8   const data = snapshot.docs.map((doc) => ({
9     id: doc.id,
10    ...doc.data(),
11  }));
12   return data;
13 }
```

Langkah 10 – API Mengambil Data Firebase

1. Edit pages/api/product.js:

```
src > pages > api > TS produkts > ...
1 import type { NextApiRequest, NextApiResponse } from "next";
2 import { retrieveProducts } from "../../utils/db/servicefirebase";
3
4 type Data = {
5   status: boolean;
6   status_code: number;
7   data: any;
8 };
9
10 export default async function handler(
11   req: NextApiRequest,
12   res: NextApiResponse<Data>,
13 ) {
14   const data = await retrieveProducts("produk");
15   res.status(200).json({ status: true, status_code: 200, data: data });
16 }
17
```

2. Jalankan browser <http://localhost:3000/api/produk>

Save	Copy	Collapse All	Expand All	Filter JSON
status:	true			
status_code:	200			
data:				
0:				
id:	"xhVPh9pDyiGP6HgL2dJP"			
name:	"baju polo"			
price:	100000			
size:	"xl"			

3. Modifikasi index.ts pada produk sesuaikan nama typenya dan db nya

```

src > pages > views > produk > index.tsx > ProdukPage > product.map() callback
1 import { useRouter } from "next/router"; // Remove this unused import
2 import { useEffect, useState } from "react";
3
4 type ProductType = {
5   id: string;
6   name: string;
7   price: number;
8   size: string;
9 }
10
11 const ProdukPage = () => {
12   // const [isLoggedIn] = useState(false); // Remove this commented code
13   // const [push] = useRouter();
14
15   const [product, setProducts] = useState<ProductType[]>([]);
16
17   // useEffect(() => { // Remove this commented out code.
18   //   if (!isLoggedIn) {
19   //     push("/views/auth/login");
20   //   }
21   // }, [isLoggedIn, push]);
22
23   useEffect(() => {
24     fetch("/api/produk")
25       .then((response) => response.json())
26       .then((responseData) => {
27         // console.log("Data produk:", responseData.data); // Remove this log
28         setProducts(responseData.data);
29       })
30       .catch((error) => {
31         console.error("Error fetching produk:", error);
32       });
33   }, []);
34
35   return (
36     <div>
37       <h1>Daftar Produk</h1>
38       {product.map((product: ProductType) => (
39         <div key={product.id}>
40           <h2>Nama: {product.name}</h2>
41           <p>Harga: {product.price}</p>
42           <p>Ukuran: {product.size}</p>
43         </div>
44       ))}
45     </div>
46   );
47 };
48
49 export default ProdukPage;

```

Jalankan browser

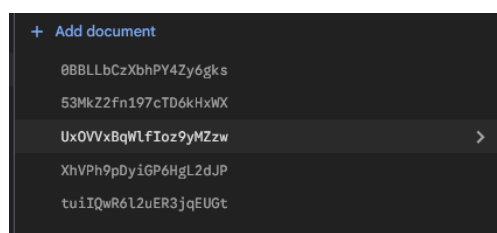


F. Tugas Praktikum

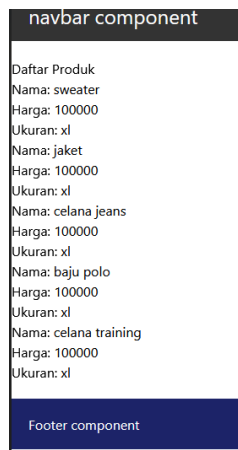
Tugas 1 (Wajib)

- Tambahkan minimal 3 data produk di Firestore
- Pastikan data tampil di halaman produk

○ Data:

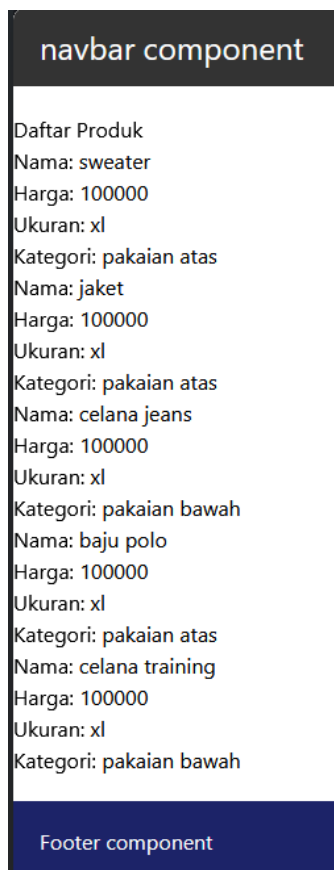


○ Web:



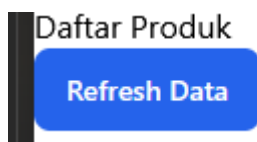
Tugas 2 (Wajib)

- Tambahkan field baru:
 - category
- Tampilkan category di frontend



Tugas 3 (Pengayaan)

- Tambahkan tombol Refresh Data
- Gunakan fetch ulang tanpa reload halaman



G. Pertanyaan Evaluasi

1. Apa fungsi API Routes pada Next.js?
= Untuk membuat backend API langsung di dalam proyek Next.js tanpa server terpisah.
2. Mengapa .env.local tidak boleh di-push ke repository?
= Karena berisi secret seperti API key yang bersifat sensitif dan rahasia.
3. Apa perbedaan data statis dan data dinamis?
= Statis di-build sekali, Dinamis di-fetch ulang setiap request.
4. Mengapa Next.js disebut framework fullstack?
= Karena bisa handle frontend (React) dan backend (API Routes) sekaligus.